20/03/2025
Thursday

Loops

i) for loop

Range function

Range function is used to generate an immutable sequence

Syntax :- range (start, stop, step)
where to start ↓ excluded by 1 ↳ By default value is 1.

It returns an immutable sequence.

- for loop is used to repeate the statements n number of times.
- It is used to traverse the sequence / data structures like list, tuple etc.

Syntax :- for variable in sequence ( ) :
statements

for example :- for i in range (1, 15, 1) :
print (i)

1
2
3
⋮
14

// output.

Ques- WAP in python to print numbers from 11 to 25.

Answer- for i in range (11, 26, 1):
print (i)

Ques- WAP in python to print sequence of number in reverse order
Answer-

for w in range (1, 101, -1):
print (w)

Ques- WAP in python to traverse into a list.
Answer-

```
list1 = [23,24,22,234,243]
for i in list1:
    print (i)
```

```
23
24
22
234
243
```

output

Ques- WAP in python to print sum of first n no. Enter by the user.
Answer-

```
sum = 0
n = int (input ("Enter a number"))
for i in range (1, n+1, n+1):
    sum = sum + i
print (sum)
```

While Loop

While loop allows a part of code to be executed as long as the given condition is true.

Syntax:- while condition (expression :
statements

for example :-

```
i = 1
while i < 5 :
    print (i)
    i = i+1
```

```
1
2
3
4
```

output

Ques- WAP in python to count the no. of digits of a number entered by the user.

Answer-
```
n = int (input ("Enter the number"))
count = 0
while (n>0):
    n = n//10
    count += count + 1
print (count)
```

Ques- WAP in python to check whether a number is an armstrong or not.

Answer-
```
n = int (input ("Enter the number:"))
b = n
sum = 0
while n>0:
    r = n%.10
    sum = sum+(r*r*r)
    n = n//10
if temp == sum:
    print ("It is an armstrong Number")
else:
    print ("It is not an armstrong Number")
```

else with loops

1. for loop / else with for loop

→ The else clause executes after the loop completes normally
→ This means that the loop did not encounter a break statement.

for example:-

Input	Output
for i in range (1, 10): print (i) else : print ("Loops Ends")	1 2 3 4 5 6 7 8 9 loop ends

Input	Output
for i in range (1, 10) : print (i) if i==5: break else : print ("Loops ends")	1 2 3 4 5

Input	Output
for i in range (1, 1): print (i) else : print ("loops ends")	Loops ends

* NOTE : If the for loop does not runs even at least once, else will be executed.

Q. Else with while loop

→ The else clause executes after the loop completes normally.
→ This means that the loop did not encounter a break statement.

for example :-

Input	Output
i = 1 while i <= 5 : print (i) i = i + 1 else : print ("loop ends")	1 2 3 4 5 loop ends
i = 1 while i <= 5 : print (i) i = i + 1 if i == 3 : break else : print ("loop ends")	1 2
i = 1 while i <= 0 : print (i) i = i + 1 else : print ("loop ends")	loops ends.

* NOTE
If the while loop does not runs even at once, else will be executed

Short Circuiting / Lazy evaluation in Python

AND

X	Y	O/P
0	0	0
0	1	0
1	0	0
1	1	1

OR

X	Y	O/P
0	0	0
0	1	1
1	0	1
1	1	1

Input	Output
i) if (True and 5/0): print ("Hi") else : print ("Dron acharya")	error

Input	Output
ii) if (True or 5/0): print ("Hi") else : print ("Dron acharya")	Hi

→ It is the act of avoiding executing parts of a boolean expression that have no effect on the final result.

Input	Output
iii) if (False or 5/0): print ("Hi") else : print ("Dronacharya")	error
iii) if (False and 5/0): print ("Hi") else : print ("Dronacharya")	Dronacharya

27/03/2025
Thursday

PAGE NO.
DATE

OS and System Module

→ OS and sys module provide numerous tools to deal with file name, path and directories.
→ OS module automatically performes many operating system tasks.
→ The OS module in python provides functions for creating and removing directories or folder, fetching its content, changing and identifying the current directory etc.

1) getcwd()

This function return the current working directory.

Syntax:
```
import os
os.getcwd()
```

2) mkdir()

This function is used to create a new directory using os.mkdir().

Syntax:
```
import os
os.mkdir()
os.mkdir("MyPythonProject")
```

3) Chdir()

This function is used to change the current working directory.

Syntax:
```
import os
os.chdir("new_data")
os.getcwd()
```

4) rmdir()

This function is used to remove a specific directory.

Syntax : import os
os.rmdir("directory_name")

List files and Sub directories

1) listdir()

This function returns the list of all the directories or files of a specific directory.

Syntax : import os
os.listdir("new_data")

2) os.name()

This function provides the name of a operating system modules.

Syntax : import os
print(os.name())

3) os.error()

→ This function defines the OS level errors.
→ It raises OS error in case of invalid or inaccessible file name and path etc.

4) os.open()

This function opens a file.

5) os.remove()
A file can be removed using os.remove().

6) os.rename()
A file and directory can be renamed by using os.rename()

Sys module

This module provides functions and variables used to mani-ulate different parts of the python run-time environm

1) Command line argument
These are arguments passed when we run a python progra / script on command line.

Syntax: python filename.py arg1 arg2 arg3

These are stored in a list name "argv" inside sys modul We can use these arguments using, indexing, like -
sys.argv[0] // for the first argument

Syntax: import sys
print (sys.argv)

2) Version
Understand the version of python

Syntax: import sys
print (sys.version)

3) Executable
It is a string that tells you where your file system or your python interpreter exists.

Syntax: import sys
print (sys.executable)

4) Platform
This variable is used to identify the host operating system.

Syntax: import sys
print(sys.platform) // print only os name

5) exit()
This function is used to terminate a program when needed

Syntax: import sys
sys.exit()

6) path
This variables is used to identify the host operating system

Syntax: import sys // prints path where
print (sys.path) python modules are searched

7) modules
Return the list of available modules

Syntax: import sys
print (sys.module)

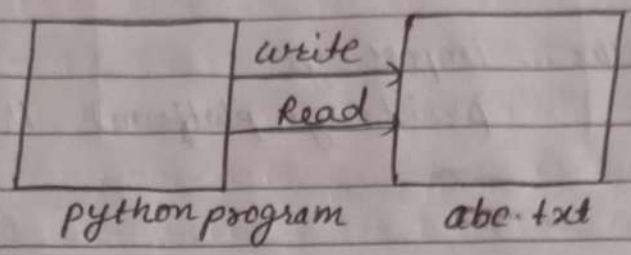
8) copyright

This displays the copyright info.

Syntax: import sys
print (sys.copyright)

File Handling

→ File is a named location on disk to store related information
→ It is used to store data in memory.
For example - **Harddisk**



There are three operations in file handling -:
1. Open a file.
2. Read / write a file.
3. close a file

1. Opening a file

Syntax - file object = open (<filename>, <accessmode>)

Example - f = open ("abc.txt", "w")

→ Python provides an open function that accepts to arguments which are filename and accessmode by which the file is accessed.
→ The function return a file object which can be used to perform various operations like writing, reading etc.

→ The file can be accessed using various modes like read, write or append.

★ The following are the details about various access modes to open file -

(1.) r (read)
It opens the file in read only mode.
The file pointer exists at the begining of the file.
The file which is opening in r mode must exist.

Example- f = open ("abc.txt", "r")

(2.) rb (read binary mode)
It opens the file in read only mode in binary format.

(3) w (write mode)
It opens the file in write mode only.
If file does not exist, then a new file is created.
If file exist new data replace old data.

Example - f = open ("abc.txt", "r")

(4) wb (write binary mode)
It open the file in write mode only in binary forma

(5) a (append)
It opens the file in append mode only.
The file pointer pointed at the end of the previous ~~return~~ written file if exist any.
If exist any
It create a file, if no file exist with same name.

⑥ r+ (read and write mode)
It open the file in read and write both modes.
The file pointer exist at the begining of the file.

⑦ w+ (write and read mode)
It open the file in write and read both modes.
It is different from r+ in the sense that it overwrite the previous file if one exist, whereas, r+ does not overwrite the previous written file.
It creates a new file if does not exist.
The file pointer exist at the begining of the file.

⑧ ab (append binary mode)
Append binary opens the file in append mode only in binary format.

⑨ a+ (append read mode)
It opens the file in append and read both modes.
The file pointer remains at the end of the file.
It creates a new file if no file exist with the same name.

{ NOTE :- If we do not mention any mode then by defaulf file opens in read only mode }

2. Closing a file

when we are done with operations on file, then we have to close the file, for that we use close function.

Example- f = open ("abc.txt", "w")
f.close()

3. The with statement

Syntax- with open ("abc.txt", "w") as f:
statements

With statement was introduced in python 2.5.
The with statement is useful in case of manipulating the file.

03/04/2025
Thursday

Writing a file

We use two function to write in a file -:
i) write ()
ii) writelines ()

Writing
To write some text into a file first we need to open the fi using the open method with 'a' & 'w' mode.

① write()
This method is used to write the string into the text file single line

Example i) f = open ("abc.txt", "w")
f.write ("python programming")
f.close()

ii) text = python programming
f = open ("abc.txt", "w")
f.write (text)
f.close ()

② writelines ()
This method is used to write multiple lines in one step

Example - f = open ("abc.txt", "w")
f . writelines (["Hi, python programming is case sensitive language"])
f . close ()

Reading a file

We can read file complete in once or we can read it in part

There are three types of Reading a file
i) read()
ii) readline ()
iii) readlines ()

① read()
This function is used to read some specific characters of a string passed in the parameter of read function.

Example - f = open ("hello.txt", "r")
print(f . read (10))
f . close()

Hi, python // output

② readline ()
This function reads a line from a file and return it as a string. (It returns one line).

Example- f = open ("hello.txt", "r")
print(f . readline ())
f . close()

Hi, python is an // output

③ readlines()
This function reads all of the lines and return them as string elements in a list. One for each line.

Example - f = open ("hello.txt", "r")
print (f.readlines())
f.close ()

Deleting a file
- To delete a file, we need to import the os module and runs its remove function.

Example - import os
os.remove ("text.txt")

Ques- WAP in python to remove / delete a file by checking whether it exist or not.
Answer - import os
if os.path.exists ("text.txt"):
os.remove ("text.txt")
else:
print ("This file does not exists")

Ques- WAP in python to display number of lines in a file.
Answer -
f = open ("text.txt", "r")
p = f.readlines()
print (len(p))
f.close ()

Ques- WAP in python to copy content of one file to another file.
Answer -

P.T.O

```
f1 = open ("abc.txt", "r")
f2 = open ("text.txt", "w")
f2.writelines (f1.readlines())
f1.close()
f2.close()
```

tell() & seek() function

① tell()

→ It is used to check current position of the file pointer.

→ It return the index position of the file pointer.

→ It takes no parameters.

Syntax - fileobject.tell()

Example - f.tell()

② seek()

→ This function is used to change the position of the file pointer to give it a specific position.

→ It takes two parameters. The second one is optional.

Syntax - fileobject.seek (offset, whence)

Example - fileobject.seek(6)

Offset

Number of index position to move forward.

Whence

→ It is an optional parameter.

→ It is used to set the point of reference to start from which place.

Whence parameter

0 → set the reference point at the begining of the file.

1 → set the reference point at the current file.

2 → set the reference point at the end of the file.

Ques- WAP in python to implement tell and seek function using python.

Answer-

```
f = open ("text.txt", "r")
print(f.tell ( ))
print(f.seek (6,1))
f.close()
```

// method 1

```
with open ("example.txt", "w") as file:
file.write ("Hello World"):
with open ("example.txt", "r") as file:
print (file.read(5))
print ("Cursor at:", tell())
seek (0)
```

08/04/2025
Tuesday

CSV files (comma Seperated values)

→ CSV stands for comma seperated value file.
→ It is the type of text file that has comma seperated struc
to arrange tabular data such as a spreadsheet or database

Ques- WAP in python to create a CSV file.

Answer-
```
import CSV
with open ("xyz.csv", "w") as CSVfile:
    x = CSV.writer (CSVfile)
    x.writerow (["1", "John"])
    x.writerow (["2", "Yashika"])
```

Reading a CSV file

→ Reading from a CSV file is done using the reader object.
→ The CSV files is opened as a text file with python's built in open function (open()), which returns a file object.
→ In this example we first open the file in read mode, file objec is converted to CSV.reader object and further operations take plac

for example -

```
import CSV
with open ("xyz.csv", "r") as f:
    a = CSV.reader (f)
```

Writing to a CSV file

→ To write to a CSV file, we first open the CSV file in write mo
→ The file object is converted to CSV.writer object and furthe operations takes place.

Ques- WAP in python to write into a CSV file entering the data of branches of a course.

Answer-
```
import CSV
with open ("abc.csv", "w") as f:
    b = CSV.writer (f)
    b.writerow (["AIML"])
    b.writerow (["CSE"])
    f.close ()
```

TSV (Tab Separated Values)

→ The TSV file stands for tab separated values file
→ It is a text file that stores data in a tabular format.
→ The TSV file format is widely used for exchanging data between databases in the form of database, table or spreadsheet data

Reading a TSV file

→ We can read a TSV file in python using open function (open()).

For example -

```
import csv
with open ("xyz.tsv") as f:
    c= csv.reader (f, delimeter="\t")
```

Writing to a TSV file

→ Now we shall see how to write over an already existing TSV file.
→ We shall make use of open function but this time we shall open the file in "wt" mode.
→ Using 'wt' mode, we can write the file as text.
→ Instead of csv.reader here we shall be using csv.writer ()
→ We shall pass the TSV file and the delimiter as ('\t') to the writer function.

Ques- WAP in python to create a TSV file and write some data in it.

Answer-

```
import csv
with open("xyz.tsv","wt") as f:
    b= csv.writer (f, delimeter="\t")
    b.writerow (["Python","C++","DataScience"])
```

Functions in Python

→ A function is a set of statements written for doing a specific task.

→ Function are used to divide a large program into smaller pieces

→ The idea is to put some commonly and repeatedly done task together and make a function, so that instead of writing the same code again and again for different inputs we can call the function.

Types of function in python

→ Predefined

→ Userdefined

Predefined

The function which are already defined in python library.

For example -

print(), max(), min(), input(), type(), len()

Userdefined

The functions which are created by the user are user defined functions.

Program Structure and Design

Creating a user-defined function

Syntax - def function_name():
statements

Example - def fun():
print("This is function")
print("This is function second line")

```
print ("start of program")
fun ()
```

Start of program
This is function
This is function second line

output

Function Arguments and Return value

→ Information can be passed into functions as arguments.
→ Arguments are specified after the function name, inside the parenthesis.

Ques- WAP in python to create a function taking arguments and returning values.

Answer-

```
def (sum (a,b):
    c = a+b
    return c
s1 = sum(5,10)
print ("Sum is ",s1)
s2 = sum (450, 320)
print ("Sum is ", s2)
```

Sum is 15
Sum is 770

output

Types of Arguments

Formal Vs Actual Argument

Formal Argument

→ When function is defined it has some parameters within the parenthesis

→ These parameters which receives the value sent from the function call are called formal argument.

Example - def my-function (a, b):

Actual Arguments

→ The parameters which we use in the function call or the parameters which we use to send the values/data during the function call are called actual arguments.

Example - my-function (5, 10)

```
def sum (a, b):
    c = a + b
    return c
```
} formal arguments

```
s1 = sum (5, 10)
print (s1)
```
} actual arguments

Named Argument

→ Python allows to pass function arguments in the form of keywords which are also called named arguments.

→ Variables in the function defination are used as keywords.

→ When the function is called you can explicity mention the name and its value.

Example -

```
def product (a, b):
    print ("Product", (a*b))
product (43, 34)
product (b=43, a=34)
```

↓

named Argument
or
Keyword Argument

Required Argument

Example -

```
def sayHello (name):
    print ("hello", name)
sayhello ("Yashika")
sayhello ("Daisy")
```

→ In this argument a required argument is a parameter th must be provided while calling a function. otherwise it will throw an error.

Keyword Argument

Keyword Argument is same as named argument.

Example -

```
def product (a, b):
    print ("Product", (a*b))
product (43,34)
product (b=43, a=34)
```

↓

Keyword argument.

Default Argument

Example-
```
def sayhello (name = "Yashika")
    print ("hello", name)
sayhello ()
```

hello yashika // output

→ It is the parameter that has a default value assigned to it.
→ If the user does not provide a value for the argument the function uses the default.

Recursive Function

→ When a function calling itself.
→ This has the benefit of meaning that you can loop through data to reach a result.

Example-

Ques- WAP to find the factorial using recurssion.

Answer-

Method-1
```
n = int (input (" Enter the value "))
if n < 0:
    print ("Factorial is not defined")
else:
    fact = 1
    for w in range (1, n+1):
    fact *= w
    print (f "The factorial {n} is {fact}")
```